

Echo State Network (ESN) — Mathematical Walkthrough of the Provided Implementation

Scope

This note explains, step by step, how the included ESN program constructs the reservoir, trains the linear readout with ridge regression, and runs inference (open/closed loop), precisely matching the implementation you provided. Key components live in: *ESN.py*, *reservoir.py*, *trainer.py*, and *utils.py*¹.

1 Model definition and notation

Shapes. Let

Base input dimension d_{in} , Output dimension d_{out} , Feedback length $f \in \{0, 1, 2, \dots\}$,
Effective input dimension $d = d_{\text{in}} + f d_{\text{out}}$, Reservoir size R , Bias column (augmented) $+ 1$.

At training time the code supports batches of size B with sequence length T ; at inference, it advances one sequence at a time².

Hyperparameters. Leak rate $\ell \in (0, 1]$, spectral radius target $\rho_\star > 0$, sparsity $s \in (0, 1]$, input/bias scalings $\alpha_{\text{in}}, \alpha_b > 0$, ridge parameter $\lambda \geq 0$, washout $t_w \geq 0$ ³.

2 Reservoir construction

Input and bias weights. The reservoir stores

$$W_{\text{in}} \in \mathbb{R}^{R \times d}, \quad (W_{\text{in}})_{ij} \sim \mathcal{U}[-\alpha_{\text{in}}, \alpha_{\text{in}}], \quad \mathbf{b} \in \mathbb{R}^R, \quad b_i \sim \mathcal{U}[-\alpha_b, \alpha_b],$$

where $\mathbf{b}$ is a per-unit bias vector added each step⁴.

Recurrent weights with target spectral radius. A dense Gaussian matrix $G \in \mathbb{R}^{R \times R}$ is drawn, elementwise masked by a Bernoulli sparsity mask M with keep-probability s ,

$$W_0 = G \odot M, \quad M_{ij} \sim \text{Bernoulli}(s).$$

Let $\rho(W) = \max_i |\lambda_i(W)|$ denote the spectral radius. The implementation rescales

$$W \leftarrow \frac{\rho_\star}{\rho(W_0)} W_0 \quad (\text{if } \rho(W_0) = 0, \text{ use } 1 \text{ to avoid division by zero}),$$

so that $\rho(W) \approx \rho_\star$ at initialization⁵.

¹Core orchestration and training/prediction: *ESN.py*. Reservoir construction/update: *reservoir.py*. Closed-form readout training & conditioning diagnostics: *trainer.py*. Utility functions (spectral radius, MAE/MSE): *utils.py*.

²Batch + time layout in *ESN.fit*: inputs $X \in \mathbb{R}^{B \times T \times d_{\text{in}}}$ are stacked; targets $Y \in \mathbb{R}^{B \times T \times d_{\text{out}}}$.

³All appear explicitly in the ESN/Reservoir/Trainer constructors and are logged; see *ESN.__init__*, *Reservoir.__init__*, *Trainer.__init__*.

⁴Uniform initializations of *W_in* and *W_bias*; *W_bias* is a squeezed vector used for broadcast addition.

⁵Random normal init, sparsity mask, compute spectral radius via *compute_spectral_radius* (max abs eigenvalue), then scale.

3 State update (teacher forcing and leaky integration)

At time t , with reservoir state $\mathbf{x}_t \in \mathbb{R}^{B \times R}$ and effective input $\mathbf{u}_t \in \mathbb{R}^{B \times d}$, the next state is

$$\mathbf{x}_{t+1} = (1 - \ell) \mathbf{x}_t + \ell \tanh(\mathbf{u}_t W_{\text{in}}^\top + \mathbf{x}_t W^\top + \mathbf{b}).$$

Here $\mathbf{b}$ broadcasts across batch⁶.

Feedback concatenation. If feedback length $f > 0$, the effective input concatenates f past outputs to the base input:

$$\mathbf{u}_t = [\mathbf{u}_t^{\text{base}} \parallel \mathbf{y}_{t-1}, \mathbf{y}_{t-2}, \dots, \mathbf{y}_{t-f}],$$

with a FIFO buffer. During training, these buffered outputs are teacher-forced targets $\mathbf{y}_t^{\text{TF}}$; during inference they are model predictions⁷.

Washout and augmented design vector. After a fixed washout t_w , the code collects an augmented state

$$\tilde{\mathbf{x}}_t = [\mathbf{x}_t \parallel \mathbf{1} \cdot \alpha_b] \in \mathbb{R}^{B \times (R+1)}, \quad t \geq t_w,$$

i.e. appending a constant bias column scaled to α_b ⁸.

4 Readout training (ridge regression)

Stack all post-washout pairs into matrices

$$X \in \mathbb{R}^{N \times (R+1)}, \quad Y \in \mathbb{R}^{N \times d_{\text{out}}}, \quad N = B(T - t_w).$$

The readout $W_{\text{out}} \in \mathbb{R}^{d_{\text{out}} \times (R+1)}$ minimizes

$$\min_{W_{\text{out}}} \|XW_{\text{out}}^\top - Y\|_F^2 + \lambda \|W_{\text{out}}\|_F^2,$$

with closed-form solution

$$W_{\text{out}} = \left((X^\top X + \lambda I)^{-1} X^\top Y \right)^\top,$$

implemented either by explicit inverse (`inv`) or via Cholesky factorization^{9, 10}.

Design-matrix health checks. The program also computes the fraction of “dead” features (near-zero variance) and saturation (share of $|x| > 0.99$)¹¹.

⁶Exact formula in `Reservoir.update_batch` with `pre = u @ W_in.T + x @ W.T + W_bias` and `x_new = (1-leak)*x + leak*tanh(pre)`.

⁷See the `prev_fb` buffer usage in `ESN.fit` (teacher forcing with `Y[:, t, :]`) and in `ESN.forward` (predicted y).

⁸The augmented vector is built via `xb = torch.cat([x, bias_vec], 1)` with `bias_vec = ones * alpha_b`.

⁹Forming $X^\top X$ and $X^\top Y$ and solving for W is done in `Trainer.fit`; `inverse` or `cholesky.solve` branches match the formula above (transpose applied at the end).

¹⁰Before/after solving, the trainer logs SVD-based diagnostics for $X^\top X$: shape, numerical rank, $\kappa = \sigma_{\text{max}}/\sigma_{\text{min}}$, and extreme singular values.

¹¹Computed in `ESN.fit` as feature variances of X and saturation fraction of the non-bias columns.

5 Inference (prediction)

Given a single input sequence $\{\mathbf{u}_t^{\text{base}}\}_{t=0}^{T-1}$:

1. Initialize $\mathbf{x}_0 = \mathbf{0}$, feedback buffer (if any) to zeros.
2. For $t = 0, \dots, T - 1$:
 - (a) **Closed-loop switch.** If `closed_loop_after_washout` is enabled, $t \geq t_w$, and $d_{\text{in}} = d_{\text{out}}$, replace the base input with the last prediction: $\mathbf{u}_t^{\text{base}} \leftarrow \mathbf{y}_{t-1}$.
 - (b) Form the effective input by concatenating feedback (if enabled) and update the state:

$$\mathbf{x}_{t+1} = (1 - \ell)\mathbf{x}_t + \ell \tanh(\mathbf{u}_t W_{\text{in}}^\top + \mathbf{x}_t W^\top + \mathbf{b}).$$

- (c) Build $\tilde{\mathbf{x}}_t = [\mathbf{x}_t; \alpha_b]$ and compute the readout:

$$\mathbf{y}_t = \tilde{\mathbf{x}}_t W_{\text{out}}^\top.$$

- (d) Update the feedback buffer with $\mathbf{y}_t$.

3. Return predictions $\{\mathbf{y}_t\}_{t=t_w}^{T-1}$.

¹²

6 Error metrics

When targets are provided during batch prediction, the program concatenates all post-washout predictions and targets, and computes:

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |\hat{y}_i - y_i|, \quad \text{MSE} = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2,$$

matching the utilities¹³.

7 Hyperparameter tuning

The `ESN.tune` helper:

1. Stacks input/target lists into tensors on the chosen device.
2. For each trial, samples $(R, \rho_\star, f, s, \ell, \alpha_{\text{in}}, \alpha_b, \lambda)$ within user-provided ranges.
3. Instantiates the ESN, fits, evaluates, and returns the scalar MAE.

¹⁴

8 End-to-end summary

1. Initialize W_{in} , $\mathbf{b}$, sparse W scaled to $\rho_\star$; set ℓ , t_w , λ ¹⁵.
2. Collect training pairs with teacher forcing; for $t \geq t_w$ form $\tilde{\mathbf{x}}_t$ and stack into X with Y ¹⁶.
3. Solve $(X^\top X + \lambda I) W_{\text{out}}^\top = X^\top Y$ (inverse or Cholesky)¹⁷.
4. Predict: advance the reservoir; after t_w , output $\mathbf{y}_t$, optionally closed loop¹⁸.

¹²All of the above is implemented in `ESN.forward`; trimming by washout occurs at the function's return.

¹³See `mean_absolute_error` and `mean_squared_error` in `utils.py`; `ESN.predict` computes metrics on concatenated sequences after washout.

¹⁴See `ESN.tune` for parameter suggestion and objective; `model.fit` is run on stacked tensors, then `model.predict` yields a trial MAE.

¹⁵Reservoir init and spectral scaling.

¹⁶Teacher forcing & augmented states in `ESN.fit`.

¹⁷Ridge solve and diagnostics in `Trainer.fit`.

¹⁸Forward pass, optional closed loop, and washout trimming in `ESN.forward`.

5. (Optional) Compute MAE/MSE¹⁹.

Minimal formula crib sheet.

State: $\mathbf{x}_{t+1} = (1 - \ell)\mathbf{x}_t + \ell \tanh(\mathbf{u}_t \mathbf{W}_{\text{in}}^\top + \mathbf{x}_t \mathbf{W}^\top + \mathbf{b})$,

Augment: $\tilde{\mathbf{x}}_t = [\mathbf{x}_t; \alpha_b]$, $X = \text{stack}_{t \geq t_w} \tilde{\mathbf{x}}_t$, $Y = \text{stack}_{t \geq t_w} \mathbf{y}_t^{\text{target}}$,

Readout: $\mathbf{W}_{\text{out}} = \left((X^\top X + \lambda I)^{-1} X^\top Y \right)^\top$,

Predict: $\hat{\mathbf{y}}_t = \tilde{\mathbf{x}}_t \mathbf{W}_{\text{out}}^\top$, return $\{\hat{\mathbf{y}}_t\}_{t=t_w}^{T-1}$,

Metrics: $\text{MAE} = \frac{1}{n} \sum_i |\hat{y}_i - y_i|$, $\text{MSE} = \frac{1}{n} \sum_i (\hat{y}_i - y_i)^2$.

¹⁹MAE/MSE utilities and their use in `ESN.predict`.